Object

- Technical Definition:
- “An Object is an Instance of a class”
- “An Object is the implementation of a class”

- In general we say :
- “ Any tangible thing for which we want to save Information”
- Now onwards we treat object technically.

Some Examples of objects

Men sandal

SHOES

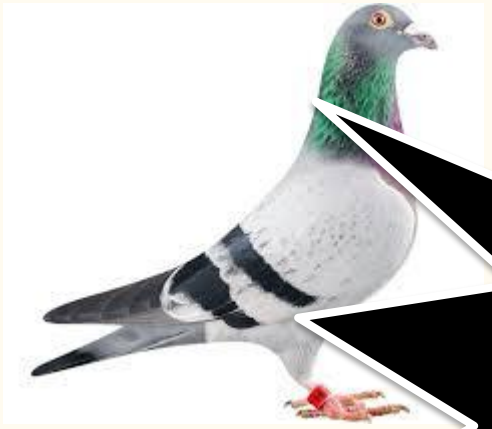
Female sandal

Loafers

Pumpies

Formal

FISH



BIRD

Animal

Discussion

- Object belongs to a group.
- Which similar.
- Have some common attributes.
- Have some common behaviors.
- We can categorized objects on some basic features .... ?

Exercise

- Are there any ‘objects’ in this classroom?

Exercise

- Are there any ‘objects’ in this classroom
  - **Chair**
  - **Multimedia**
  - **Student**
  - **Teacher**

Exercise

- Possible Objects in this university?

Class

- Collection of Similar object.
- The objects that share some common features.
- It is the a design of an object.
- It is a detail of an object.
- It tell us what an object contains in it.

Technical Definition:

- “ A class is blueprint of an object”

Summarize

- A class :
  - It’s a blue print .
  - It’s a design or template.

An Object:

- Its an instance of a class.
- Implementation of a class.

NOTE: Classes are invisible, object are visible

Class

- Accelerator Pedal
- Brake Pedal
- Steering Wheel

- User-friendly “interfaces” to control the car
- Their mechanism is *housed* inside the engineering drawings or blueprints

Class

- **Suppose you want to drive a car and make it go faster by pressing down on its accelerator pedal.**
- **What must happen before you can do this?**

Class

- **before you can drive a car, someone has to *design* it and *build* it.**
- **A car typically begins as engineering drawings**
- Unfortunately, you cannot drive the engineering drawings of a car—before you can
- drive a car, it must be built from the engineering drawings that describe it
- A completed car will have an actual accelerator pedal to make the car go faster. But even that’s not
- enough—the car will not accelerate on its own, so the driver must press the accelerator
- pedal to tell the car to go faster.

Now let’s use our car example to introduce the key object-oriented programming concepts

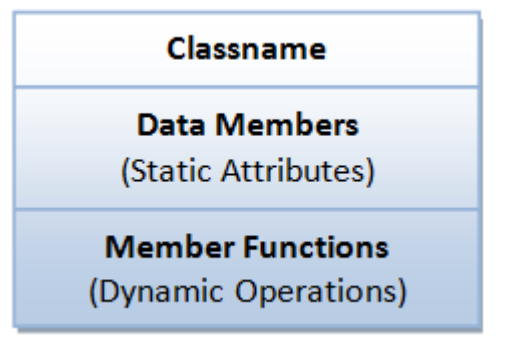
- Performing a task in a program requires a function
- The function hides from its user the complex tasks that it performs, just as the accelerator
- pedal of a car hides from the driver the complex mechanisms of making the car go
- Faster
- In C++, we begin by creating a program unit called a class to house a function, just
- as a car’s engineering drawings house the design of an accelerator pedal.

Class

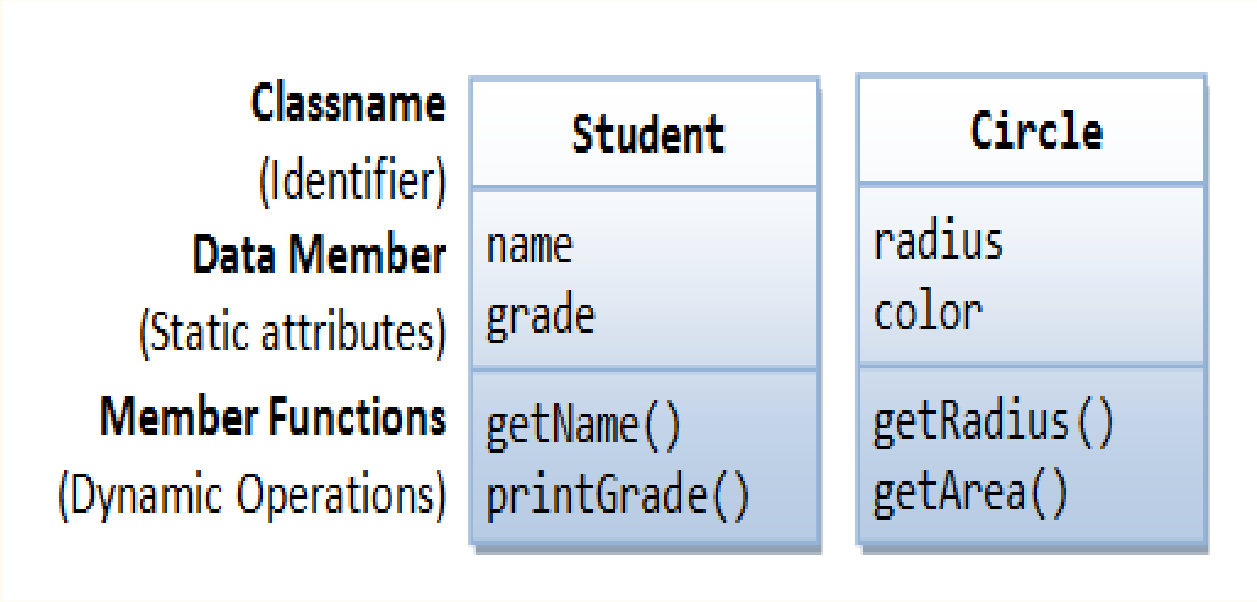
- Just as you cannot drive an engineering drawing of a car, you cannot “drive” a class.
- Just as someone has to build a car from its engineering drawings before you can actually
- drive the car, *you must create an object of a class before you can get a program to perform the*
- *tasks the class describes*.
- Note also that just as *many* cars can be built from the same engineering
- drawing, *many* objects can be built from the same class

A Class is a 3-Compartment Box encapsulating Data and Functions

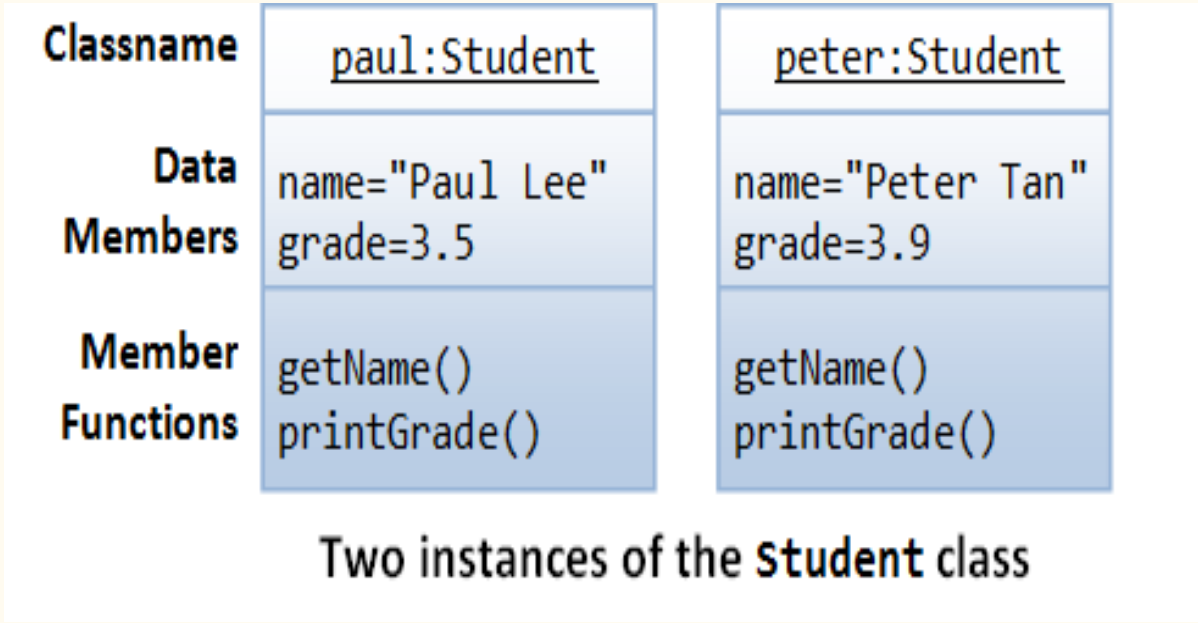
1.Classname (or identifier): identifies the class.
2.Data Members or ***Variables*** (or *attributes*, *states*, *fields*): contains the *attributes* of the class.
3.Member Functions (or *methods*, *behaviors*, *operations*): contains the *dynamic operations* of the class.



Example of classes



Example of objects



Each object of a class maintains its own copy of its attributes in memory

Classes

```
class Car
{
        void accelerate()
        { \\ logic for acceleration }

        void brake()
        { \\ logic for brakes }
};
```

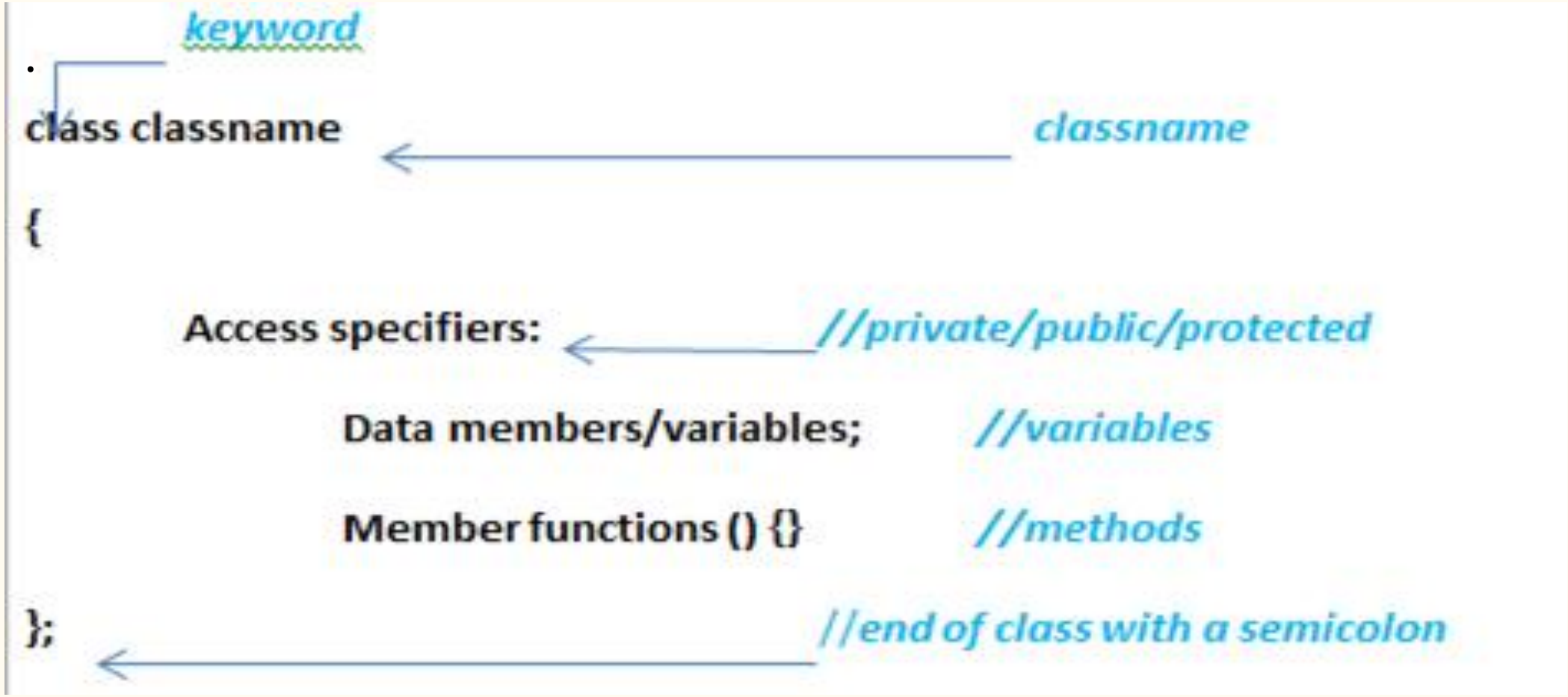
classes

```
class Car
{
        string model;
        int numOfDoors;
        string color;

        void accelerate()
        { \\ logic for acceleration }

        void brake()
        { \\ logic for brakes }

};
```

How to use the car?

```
int main()
{
        Car car;
        car.accelerate();
}
```

Syntax of Class



Sructure

- **structure** is another user defined data type available in C/C++ that allows to combine data items of different kinds.
- Structures are used to represent a record. Suppose you want to keep track of your books in a library. You might want to track the following attributes about each book −
  - Title
  - Author
  - Subject
  - Book ID

Syntax of structure

- The struct keyword defines a structure type followed by an identifier (name of the structure).
- Then inside the curly braces, you can declare one or more members (declare variables inside curly braces) of that structure.
  - For example:

```
struct Person
{
    char name[50];
    int age;
    float salary;
};
```

C Structure vs C++ Structure

1. Member functions inside structure:

- Structures in C cannot have member functions inside structure but Structures in C++ can have member functions along with data members.

```
struct person_str {
string name; int age;
//constructor
person_str() {
name = "default";
age = 77;}
```

C Structure vs C++ Structure

2. Direct Initialization:

- We cannot directly initialize structure data members in C but we can do it in C++

```
#include <stdio.h>

struct Record {
        int x = 7;
};

// Driver Program
int main()
{
        struct Record s;
        printf("%d", s.x);
        return 0;
}
```

```
/* Output : Compiler Error
6:8: error: expected ':', ',', ';', '}' or
'__attribute__' before '=' token
int x = 7;
          ^
In function 'main': */
```

C Structure vs C++ Structure

3. Using struct keyword:

In C, we need to use struct to declare a struct variable. In C++, struct is not necessary. For example, let there be a structure for Record. In C, we must use “struct Record” for Record variables. In C++, we need not use struct and using ‘Record‘ only would work.

```
struct Record {
    int x = 7;
};

// Driver Program
int main()
{
    struct Record s;
    printf("%d", s.x);
    return 0;
}
```

```
#include <iostream>
using namespace std;

struct Record {
    int x = 7;
};

// Driver Program
int main()
{
    Record s;
    cout << s.x << endl;
    return 0;
}
```

C Structure vs C++ Structure

4. **Static Members:** C structures cannot have static members but is allowed in C++.
5. **Access Modifiers:** C structures do not have access modifiers as these modifiers are not supported by the language. C++ structures can have this concept as it is inbuilt in the language.
6. **Constructor creation in structure:** Structures in C cannot have constructor inside structure but Structures in C++ can have Constructor creation.

Structure vs Classes

- In C++, a structure is the same as a class except for a few differences.
- The most important of them is security.
- A Structure is not secure and cannot hide its implementation details from the end user while a class is secure and can hide its programming and designing details.
  - **Following are the points that expound on this difference:**
  - What if we forget to put an access modifier before the first field?

1) Data Members of a class are private by default and members of a struct are public by default.

```
struct Robot {     OR     class Robot {
  float locX;                 float locX;
```

Structure vs Classes

- **Class**

Classes are of reference types.

All the reference types are allocated on heap memory.

Class has limitless features.

Class is generally used in large programs.

A Class can inherit from another class.

- **Structure**

Structs are of value types.

All the value types are allocated on stack memory.

Struct has limited features.

Struct are used in small programs.

A Struct is not allowed to inherit from another struct or class.